

NETFLIX PRIZE RECOMMENDATION SYSTEM

Personalized Content Discovery using Collaborative Filtering and Matrix Factorization

1. PROBLEM UNDERSTANDING

The rapid growth of digital streaming platforms has created an information overload problem, where individuals are faced with thousands of content items but have no proper means of finding relevant content. Recommendation engines tackle this issue by understanding user preferences automatically and providing personalized recommendations.

This project aims at developing a recommendation engine based on the Netflix Prize Dataset, which is one of the most popular datasets used in recommendation system research. This dataset contains 100,480,507 ratings provided by 480,189 users to 17,770 movies, thus offering a perfect case study for recommendation research.

The key problems of the project include:

- Identifying user preferences through sparse interaction history
- Predicting ratings of unseen movies by users
- Generating Top-10 recommendations for individual users
- Dealing with extremely sparse data (missing 99.4% of ratings)
- Evaluating different recommendation strategies.

The significance of this project lies in its business impact, which comes from improving customer experience and increasing profits due to higher engagement rates. For instance, Netflix estimated savings of \$1 billion per year from reduced churn.

2. EXPLORATORY DATA ANALYSIS

2.1 Dataset Overview

The full Netflix Prize Dataset contains 100M+ ratings. For this project, a subset was created by keeping users who rated between 50 and 200 movies, resulting in:

- Total ratings: 18,829,034
- Total users: 176,959
- Total movies: 17,767
- Sparsity: 99.40%

This means only 0.6% of all possible user-movie pairs have ratings, making this a highly sparse matrix — a fundamental challenge for recommendation systems.

2.2 Rating Distribution

Ratings are left-skewed — 61% are 4-5 stars showing users rate movies they enjoyed. Average user rated 106 movies. Top rated movie is Miss Congeniality (~85,000 ratings).

2.3 User Activity

- Average ratings per user: 106
- Median ratings per user: 97
- Min ratings per user: 50
- Max ratings per user: 200

Users in this subset are moderately active — sufficient interaction history for collaborative filtering to work effectively.

2.4 Content Popularity

Top 10 most rated movies are dominated by mainstream Hollywood films:

- Miss Congeniality (~85,000 ratings)
- Independence Day (~73,000 ratings)
- The Day After Tomorrow (~71,000 ratings)

Such a power-law distribution of recommendations (with only a few movies getting all the recommendations) is seen in actual recommendation datasets, making it difficult to recommend long-tail items.

2.5 Sparsity Analysis

The extremely sparse nature (99.4% sparsity) of such data poses an issue where similarity measures cannot work reliably because most of the movies do not have any similar users rating them.

3. METHODOLOGY

3.1 Data Preprocessing

- Loaded 4 raw rating files (combined_data_1-4.txt)
- Parsed custom Netflix format (movie_id: followed by ratings)
- Loaded movie metadata with manual CSV parsing (titles contain commas)
- Filtered users with 50-200 ratings to reduce sparsity
- Split data: 80% training, 20% testing (random_state=42)
- Used 1,000,000 rating sample for model training due to RAM constraints

3.2 Train-Test Split

- Method: random 80/20 split using Surprise library
- Training set: 800,000 ratings
- Test set: 200,000 ratings
- random_state=42 ensures reproducibility

3.3 Relevance Definition for MAP@10

As per problem statement, a movie is considered relevant if: actual user rating ≥ 3.5 stars. This threshold captures movies the user genuinely enjoyed (4-5 stars) while excluding neutral or negative experiences (1-3 stars).

4. MODEL DESIGN

4.1 Model 1 — Item-Based Collaborative Filtering

Methodology:

Item-Based CF finds movies similar to those a user has already rated and recommends the most similar unrated movies.

Parameters:

- Similarity metric: Cosine similarity
- k (nearest neighbors): 30
- Algorithm: KNNWithMeans (accounts for user rating bias)

Justification:

Item-Based CF is intuitive and explainable — "recommended because you watched X". It serves as a strong baseline and is well-studied in recommendation literature.

Limitation:

Problems caused by sparsity — due to the lack of common raters between movies, similarity calculations are inaccurate and result in overestimated recommendations (all 5.0).

4.2 Model 2 — SVD (Matrix Factorization)

Methodology:

SVD decomposes the sparse user-item matrix into latent factor representations, capturing hidden patterns in user preferences.

Parameters:

- n_factors: 100 (latent factors)
- n_epochs: 20 (training iterations)
- lr_all: 0.005 (learning rate)
- reg_all: 0.02 (regularization)

Justification:

SVD handles sparsity much better than CF by learning compact latent representations. It was used by winning teams in the original Netflix Prize competition and consistently outperforms memory-based methods on sparse datasets.

5. EVALUATION METRICS

5.1 RMSE (Root Mean Squared Error)

Purpose: Measures rating prediction accuracy

Interpretation: Lower is better.

Penalizes large errors more heavily than small ones.

5.2 MAE (Mean Absolute Error)

Purpose: Measures average prediction error

Interpretation: Lower is better

More interpretable than RMSE — directly in rating units

5.3 MAP@10 (Mean Average Precision @ 10)

Purpose: Measures ranking quality of top 10 recommendations

Interpretation: Higher is better (0 to 1)

Relevance threshold: actual rating ≥ 3.5

Procedure:

1. For each user, generate Top-10 predicted movies
2. Check which recommended movies are actually relevant (≥ 3.5)
3. Compute Average Precision considering rank position
4. Average across all users.

Trade-off:

RMSE measures exact prediction accuracy while MAP@10 measures ranking quality. A model can have poor RMSE but good MAP@10 if it correctly ranks relevant items even with inaccurate scores. This is demonstrated by Item-CF in our results.

6. EXPERIMENTAL RESULTS

6.1 Model Performance

Model	RMSE	MAE	MAP@10
Item-Based CF	1.1195	0.8630	0.9376
SVD	1.0373	0.8252	0.9383

SVD outperforms Item-Based CF on all three metrics.

6.2 SVD Error Distribution

- Mean absolute error: 0.8252 stars
- Median absolute error: 0.7017 stars
- 36.5% predictions within 0.5 stars
- 67.2% predictions within 1.0 stars
- 32.8% predictions with error > 1.0 star

6.3 Model Comparison

Recommendation Quality:

SVD wins — lower RMSE, MAE and higher MAP@10

Training Complexity:

Item-CF: $O(n^2)$ similarity computation

SVD: Matrix decomposition with controllable complexity via `n_factors`

SVD scales better with dataset size

Computational Efficiency:

Item-CF crashed on full 18.8M dataset, required 1M sample

SVD handles large sparse matrices efficiently

SVD is more memory efficient in practice

Practical Usability:

Item-CF: highly explainable ("recommended because you watched X")

SVD: black box but more accurate, used in production systems

Item-CF better for transparency, SVD better for accuracy.

7. RECOMMENDATION EXAMPLES

7.1 SVD Recommendations for User 814701

User rating history:

1★: 6 ratings | 2★: 17 | 3★: 24 | 4★: 45 | 5★: 46

Top 10 SVD Recommendations:

TITLE	PREDICTED
Little House on the Prairie: Season 2	4.94
Blue Streak	4.80
Shall We Dance?	4.79
As Time Goes By: Series 9	4.77
Patch Adams	4.75
Slayers DVD Collection	4.74
Mark Twain	4.74
Lethal Weapon 3	4.73
The Rainmaker	4.72
As Time Goes By: Series 7	4.72

7.2 Recommendation Validation

Validated SVD recommendations against test set:

Title	Predicted	Actual	Match
Soul Plane	3.01	4.0	✓

7.3 Side by Side Comparison

Both models recommended similar movies for same user showing strong agreement and validating recommendation quality. 7 out of 10 recommendations overlap between both models.

7.4 Success Cases

Predictions with error < 0.3 stars — model learned user preferences very accurately for these cases.

7.5 Failure Cases

Predictions with error > 2.0 stars — model struggled when:

- User has very few ratings (cold start)
- Movie has very few ratings (data sparsity)
- User preference deviates from similar users (individual exception)

8. Key Insights

1. Sparsity is the core challenge

99.4% sparsity makes direct similarity computation unreliable. SVD handles this better through latent factor learning.

2. SVD is the superior model

Wins on RMSE (1.0373 vs 1.1195), MAE (0.8252 vs 0.8630) and MAP@10 (0.9383 vs 0.9376).

3. Both models rank relevant content well.

MAP@10 > 0.93 for both models shows collaborative filtering approaches are highly effective at identifying relevant content regardless of exact prediction accuracy.

4. Rating bias affects predictions

61% of ratings are 4-5 stars creating positive bias. Models tend to overpredict ratings slightly.

5. Item-CF limitation is informative

Item-CF producing all 5.0 predictions proves that memory-based methods fail under extreme sparsity — directly motivating latent factor approaches like SVD.

6. Mainstream content dominates

Miss Congeniality, Independence Day dominate ratings showing popularity bias — models may over-recommend popular movies.

7. Agreement validates quality

Both models recommending same movies for same user strongly validates the recommendation quality without ground truth.

9. Future Improvements

1. Neural Collaborative Filtering

Deep learning approach capturing non-linear user-item interactions. Expected to improve RMSE below 0.90.

2. Hybrid Recommendation System

Combine SVD with content-based filtering using movie metadata (genre, year, cast) to improve cold start handling

3. Cold Start Strategy

New user: popularity-based recommendations + onboarding survey

New movie: content-based filtering using metadata

4. Hyperparameter Tuning

GridSearchCV on SVD parameters (n_factors, n_epochs, lr_all)

Expected 5-10% RMSE improvement

5. Ensemble Methods

Combine predictions from multiple models

Weighted average of Item-CF and SVD predictions

6. Diversity Metrics

Add coverage and diversity metrics to avoid filter bubbles

Ensure recommendations span different genres

7. Real-time Deployment

Flask/FastAPI REST API for real-time recommendations

Scalable architecture using approximate nearest neighbors

8. Explainability

Add explanation layer to SVD recommendations

"Recommended because users similar to you enjoyed this"